

深圳大学实验报告

课程名称 机器学习

项目名称 实验一：PCA 算法

学 院 计算机与软件学院

专 业 软件工程（腾班）

指导教师 赖志辉

报 告 人 洪子敬 学号 2022155033

实验时间 2024 年 3 月 8 日至 2024 年 3 月 17 日

实验报告提交时间 2024 年 3 月 13 日

教务处制

一、实验目的与要求

1. 掌握 PCA 算法的原理和算法实现；
2. 编码实现 PCA 算法；
3. 用 PCA 算法和最邻近算法实现人脸识别；
4. 在了解经典 PCA 算法的基础上思考 PCA 的改进算法。

二、实验内容与方法

1. 简述 PCA 的原理并证明其投影矩阵等价于重构误差最小的模型的解；
2. 实现 PCA 算法的人脸重构，即用 20, 40, 60, 80, ..., 160 个投影来重构图像的效果；
3. 使用 PCA 算法和最近邻算法进行人脸识别，给出 10, 20, ..., 160 维的人脸识别识别率；
4. 用 PCA 用来进行人脸 (AR) 与旋转目标 (COIL) 图像降维，对 2 个不同数据集 1-2 个类中图像实现二维和三维空间数据的可视化；
5. 同时设计一个新的 PCA 算法，并与经典 PCA 比较。

三、实验步骤与过程

1. PCA (Principal Component Analysis) 原理简介

(1) 基本内容：

PCA 主成分分析，是一种使用较为广泛的数据降维算法，主要思想是将 n 维数据特征映射到 k 维上，这 k 维全新的正交数据特征称为主成分；

PCA 的工作就是从原始的空间中顺序地找一组相互正交的坐标轴，新的坐标轴的选择与数据本身是密切相关的。其中，第一个新坐标轴选择是原始数据中方差最大的方向，第二个新坐标轴选取是与第一个坐标轴正交的平面中使得方差最大的，第三个轴是与第 1, 2 个轴正交的平面中方差最大的。依次类推，可以得到 n 个这样的坐标轴。通过这种方式获得的新的坐标轴我们发现，大部分方差都包含在前面 k 个坐标轴中，后面的坐标轴所含的方差几乎为 0。于是，我们可以忽略余下的坐标轴，只保留前面 k 个含有绝大部分方差的坐标轴。事实上，这相当于只保留包含绝大部分方差的维度特征，而忽略包含方差几乎为 0 的特征维度，实现对数据特征的降维处理。

(2) 算法流程：

- I. 把每个数据拉成列向量或者行向量，并组装成一个矩阵；
- II. 计算数据矩阵的协方差矩阵 C ；
- III. 对协方差矩阵进行分解求得对应的特征值和特征向量，并将特征值降序排列；
- IV. 选取特征值最大的前 p 个特征向量组成特征矩阵 P 。

Ps: 值得注意的是并不是所有矩阵都可以用**特征值分解**（即对角化协方差矩阵），必须要求有 n 个线性无关的向量，如果不满足这种情况怎么办呢，这时候我们可以采用**奇异值分解**的方法（SVD），此方法可对所有矩阵进行分解，但复杂情况视情况而定。

(3) 具体流程：

输入：数据集 $X = \{x_1, x_2, x_3, \dots, x_n\}$ ，需要降维到 k 维；

- I. 样本中心化（即每一个特征都减去各自的均值）；

II. 计算协方差矩阵 $C = X * X^T / n$ (实际上除 n 或者 $n-1$ 都一样, 因为样本数据较大时两者比较接近, 也可以不除, 因为同比例扩大缩小效果均一样);

III. 对协方差矩阵进行对角化, 并将对角线上的特征值从大到小进行排序, 选择其中 k 个最大的, 将该 k 个特征向量分别作为特征向量矩阵 w ;

IV. 用 $Y = X * w$ 就得到了降维后的新空间; (至于左乘还是右乘取决于数据是行向量还是列向量, 列向量就变成左乘 w 的转置)。

(4) 最小重构误差

之前我们是从最大化方差投影方差的角度来选取前 p 个最大特征值的特征向量组成投影矩阵, 由于之前是解决 PCA **最大可分性问题**, 这次我们从最小重构误差的角度入手, 即 PCA 的**最近重构性问题**。

首先假设数据集为 n 维, 共有 m 组数据 $(x^{(1)}, x^{(2)}, x^{(3)}, \dots, x^{(m)})$, 注意这里每一个 $x^{(i)}$ 均为 n 维的列向量; 原坐标系为 $(w^{(1)}, w^{(2)}, w^{(3)}, \dots, w^{(m)})$, 其中 w 为标准正交基, 即 $\|w\|_2 = 1$, $w^T * w = 0$;

将数据从 n 维降到 p 维, 则新的坐标系为 $(w^{(1)}, w^{(2)}, w^{(3)}, \dots, w^{(p)})$, 样本点 $x^{(i)}$ 在 p 维坐标系里的表示为 $z^{(i)} = (z_1^{(i)}, z_2^{(i)}, z_3^{(i)}, \dots, z_p^{(i)})^T$, 其中 $z_j^{(i)} = w_j^T * x^{(i)}$;

若要恢复数据, 则 $x'^{(i)} = \sum_{j=1}^p z_j^{(i)} * w_j = W * z^{(i)}$, 同理 $z^{(i)} = x^{(i)} * W$;

需要注意的是 X 是 $n * m$ 的矩阵, W 是 $m * p$ 的矩阵, 此处均采用的是列向量;

而 PCA 的目的是找到一个超平面使得所有的样本到此超平面的距离最小, 即最小化重构误差 $\sum_{i=1}^m \|x'^{(i)} - x^{(i)}\|_F^2$, 具体求解过程如下所示:

$$\begin{aligned} \text{原式} &= \sum_{i=1}^m \|W z^{(i)} - x^{(i)}\|_F^2 \\ &= \sum_{i=1}^m (W z^{(i)})^T (W z^{(i)}) - 2 * \sum_{i=1}^m (W z^{(i)})^T x^{(i)} + \sum_{i=1}^m x^{(i)T} x^{(i)} \\ &= \sum_{i=1}^m z^{(i)T} z^{(i)} - 2 * \sum_{i=1}^m z^{(i)T} W^T x^{(i)} + \sum_{i=1}^m x^{(i)T} x^{(i)} \\ &= -\sum_{i=1}^m z^{(i)T} z^{(i)} + \sum_{i=1}^m x^{(i)T} x^{(i)} \\ &= -\text{tr}(W^T (\sum_{i=1}^m x^{(i)} x^{(i)T}) W) + \sum_{i=1}^m x^{(i)T} x^{(i)} \\ &= -\text{tr}(W^T X X^T W) + \sum_{i=1}^m x^{(i)T} x^{(i)} \end{aligned}$$

由于 $\sum_{i=1}^m x^{(i)T} x^{(i)}$ 表示的是 X 的协方差矩阵(这里除不除样本总量效果都一样)

最小化上式等效于

$$\text{argmin}_W -\text{tr}(W^T X X^T W) \text{ s.t. } W^T W = I$$

利用拉格朗日乘子法有

$$J(W) = -\text{tr}(W^T X X^T W + \lambda(W^T W - I))$$

对上面函数的 W 求导得

$$-(-2X X^T W + 2\lambda W)$$

由于我们要求最小值，则令其导数值为 0，整理可得

$$XX^T W = \lambda W$$

由此可知最小值对应的 W 由协方差矩阵 XX^T 对应的 p 个最大的特征值对应的特征向量组成，至于为什么是最大的，下面给出解释：

从本质上说**特征向量**其实表示的是**数据不变的特征方向**，而**特征值**表示数据在**这个方向变化的幅度**，也就是**该方向上的方差**；而协方差矩阵我们选择 p 个最大的特征值，也就是保留了变化最显著的 p 个方向，这样我们可以最大程度地保留数据的重要信息，同时去除一些不必要的，次要的特征，从而达到**降噪**的目的。

至此我们可以发现我们**重构误差最小的模型的解恰好就是投影矩阵 W** ，通过 $W^T x^{(i)}$ 我们可以求得降维后的数据。

2. PCA 的编码实现 (matlab)

为了是整个编码过程简洁明朗，我们采用编写函数的方式进行封装处理，这样我们只需要在主函数里面对参数进行调整即可运行不同的数据集。

PCA 部分主要分成两部分，读取数据集函数 Read_data 和提取特征值函数 PCA_fuc；

对于读取数据集函数：就是将数据集读入并拉成**列向量**组成输入矩阵，并将数据按输入比例划分为对应的训练集和测试集，同时生成对应的标签数据（以便后面的人脸识别环节）。

对于提取特征值函数：就是按照 PCA 的原理和算法流程，对数据进行特征提取并返回排序好的特征矩阵。

具体函数（具体代码见压缩包）如下所示：

```
% input: [Data set address, number of categories,  
          number of each category, proportion of training sets, file format]  
% output: {training set, testing set, label of training, label of testing,  
          row of image, column of image}  
function [X_train, X_test, Y_train, Y_test, row, col]=Read_data(address, class_num, each_class_num, train_num, pattern)  
  
% input: the data to be reduced in dimension  
% output: return the sorted feature vector and the mean of the data set  
function [sorted_vectors, Mean]=PCA_fuc(X_train)
```

洪子敬 腾班
2022155033
2024.3.13 11:32

洪子敬 腾班
2022155033

主函数中代码如下所示：

```
clear;  
address='D:\机器学习\人脸数据集\AR_Gray_50by40\AR0'; % address='D:\机器学习\人脸数据集\coil-20-proc\obj';  
class_num=40;  
each_class_num=10;  
train_num=7;  
pattern='.tif'; %pattern='.png';  
% read data  
[X_train, X_test, Y_train, Y_test, row, col]=Read_data(address, class_num, each_class_num, train_num, pattern);  
% Feature extraction  
[sorted_eig_vectors, mean]=PCA_fuc(X_train);  
  
% reconstruct the face  
centered_face=(X_train-mean);  
face=X_train(:, 2);  
Rebuild(sorted_eig_vectors, face, row, col, mean);
```

洪子敬 腾班
2022155033
2024.3.13 11:34

3. 使用 PCA 进行人脸重构（分别用 20, 40, 60, 80, ..., 160 个投影来重构图像）

由于我们本实验是将图像数据拉成列向量来进行处理，而我们之前已经求出了排序好的特征向量组即投影矩阵，所以我们可以通过 $features' * face$ 来实现对人脸的降维，接着为了

重构人脸，我们用此降维后的数据再左乘 features 即可重构原始图像；我们可以理解为先求出目标人脸在特征矩阵中各个向量所占比重，在乘以特征矩阵就可以实现不同向量的加权和。此处我们也是封装一个函数 Rebuild，但我们是进行对照实验，采用不同数量的投影来进行重构人脸，从 20 到 160，以 20 为步长进行递增，最后将所有重构人脸以分成子图的形式进行展示，具体展示如下：

```
% input: {Dimensionality reduction matrix,Face to be reconstructed,
%         row of image,column of image,mean of training set}
function Rebuild(features, face, row, col, mean)
```

洪子敬 腾班
2022155033
2024.3.13

在主函数中代码如下：

```
% reconstruct the face
centered_face=(X_train-mean);
face=X_train(:,2);
Rebuild(sorted_eig_vectors, face, row, col, mean);
```

洪子敬 腾班
2022155033
2024.3.13

（这里是在训练集选了一张人脸）

在旋转目标（COIL）数据集和人脸（AR_Gray_50by40）数据集的效果如下所示：



根据结果我们可以看到随着投影维度的不断增加，重构的人脸图像越来越清晰，越来越与原来的图像逼近，效果还是很不错的。

4. 使用 PCA 算法和最近邻算法进行人脸识别（计算 10,20,...,160 维的人脸识别识别率）

在我们已经计算得到了投影矩阵之后，使用测试集进行人脸识别，通过不同维度的降维矩阵来对测试数据进行降维，降维后通过计算每个测试数据与降下相应维度的训练数据进行距离计算（这里采用欧氏距离，也可以采用曼哈顿距离），通过计算得到最短的距离对应的训练样本，获取该训练样本对应的标签数据，最后通过比较该标签数据与测试集上该样本的标签是否相同，相同则计入正确识别的数量之中，并计算识别率并将其返回给主函数，此功能通过函数 KNN 实现。KNN 函数主要是接受主函数已经传来的不同维度的降维矩阵，并进行识别功能，而主函数中主要是传递 10,20,...,160 维的降维矩阵给该函数进行降维识别。

KNN 函数如下所示：

```
% Nearest neighbor algorithm
% input: {Dimensionality reduction matrix,training set,testing set,label of
%         training,label of testing,mean of training set}
% output:the recognition rate of face recognition
function rate=KNN(features, X_train, X_test, Y_train, Y_test, mean)
```

洪子敬 腾班
2022155033
2024.3.13

在主函数代码如下所示：

```
% Face recognition
for i=10:10:160
    chosed_features= sorted_eig_vectors(:,1:i);
    rate=KNN(chosed_features,X_train,X_test,Y_train,Y_test,mean);
    fprintf("dimension=%d, success rate=%f\n",i,rate);
end
```

洪子敬 腾班
2022155033
2024.3.13

而针对旋转目标（COIL）数据集和人脸（AR_Gray_50by40）数据集的识别率结果如下所示：

dimension=10, success rate=0.966667	dimension=10, success rate=0.841667
dimension=20, success rate=0.983333	dimension=20, success rate=0.958333
dimension=30, success rate=0.983333	dimension=30, success rate=0.958333
dimension=40, success rate=0.983333	dimension=40, success rate=0.950000
dimension=50, success rate=0.983333	dimension=50, success rate=0.958333
dimension=60, success rate=0.983333	dimension=60, success rate=0.958333
dimension=70, success rate=0.983333	dimension=70, success rate=0.958333
dimension=80, success rate=0.983333	dimension=80, success rate=0.958333
dimension=90, success rate=1.000000	dimension=90, success rate=0.958333
dimension=100, success rate=1.000000	dimension=100, success rate=0.950000
dimension=110, success rate=1.000000	dimension=110, success rate=0.950000
dimension=120, success rate=1.000000	dimension=120, success rate=0.950000
dimension=130, success rate=1.000000	dimension=130, success rate=0.950000
dimension=140, success rate=1.000000	dimension=140, success rate=0.950000
dimension=150, success rate=1.000000	dimension=150, success rate=0.950000
dimension=160, success rate=1.000000	dimension=160, success rate=0.950000

洪子敬 腾班
2022155033
2024.3.12
21:27

洪子敬 腾班
2022155033
2024.3.13

由上述结果我们不难看出，针对旋转目标（COIL）数据集效果很好，识别率很高，特别是当降维矩阵维度达到 100 维左右，识别率已经差不多到 100%了，这与该数据集的图像类型有关，该数据集一共 20 类，且每一类均不同，每一类的特征都较为明显，当特征较少时，识别率也能很高，那么当给的降维矩阵维度达到 100 及以上是，特征分辨就十分明显了，基本哪一类都可以分辨出来了。即使其进行旋转，但特征向量仍十分相近，还是容易识别出来。

而针对人脸（AR_Gray_50by40）数据集，效果也还不错，但还不是特别好，最高达到 95.83%，这是因为这份数据集是人脸数据集，每类人脸都有一些相近的特征向量，因此当特征维度比较少时，识别率并不是很高，但也有 84%左右，说明我们这个算法设计的还是比较成功的。

5. 用 PCA 用来进行人脸（AR）与旋转目标（COIL）图像降维，对 2 个不同数据集 1-2 个类中图像实现二维和三维空间数据的可视化；

在前面我们已经计算出了降维矩阵，所以我们只需要将对应的数据集进行降维之后取降维后的前两个维度进行画图就得到了二位数据的可视化，同理我们取前三个维度进行画图就得到了三维空间数据的可视化。这里我们需要用到matlab中自带的画图工具，二维用plot函数，三维用plot3函数，而为了使代码更加分明，我们将两者均包装成函数，二维可视化函数TwoDimensionVisualize，三维可视化函数ThreeDimensionVisualize。

这里为方便，按照每类取7张图片进行数据的可视化，函数如下所示：

```
% input: {The arranged feature vectors, data set, mean of the data set}
% Effect: shows an image of points where the data is reduced to two dimensions
function fig=TwoDimensionVisualize(features,X,mean)

% input: {The arranged feature vectors, data set, mean of the data set}
% Effect: shows an image of points where the data is reduced to three dimensions
function ThreeDimensionVisualize(features,X,mean)
```

洪子敬 腾班
2022155033
2024.3.13

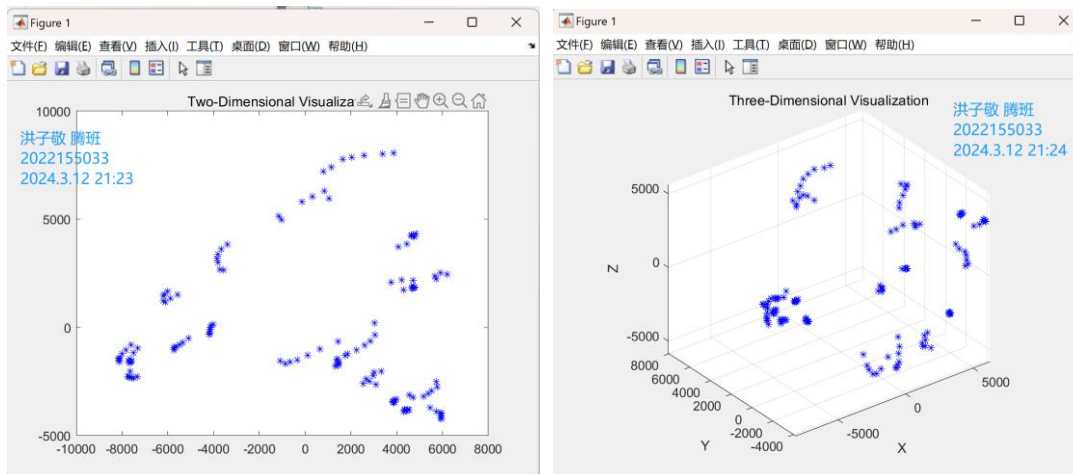
洪子敬 腾班
2022155033
2024.3.13

在主函数中的代码如下所示：

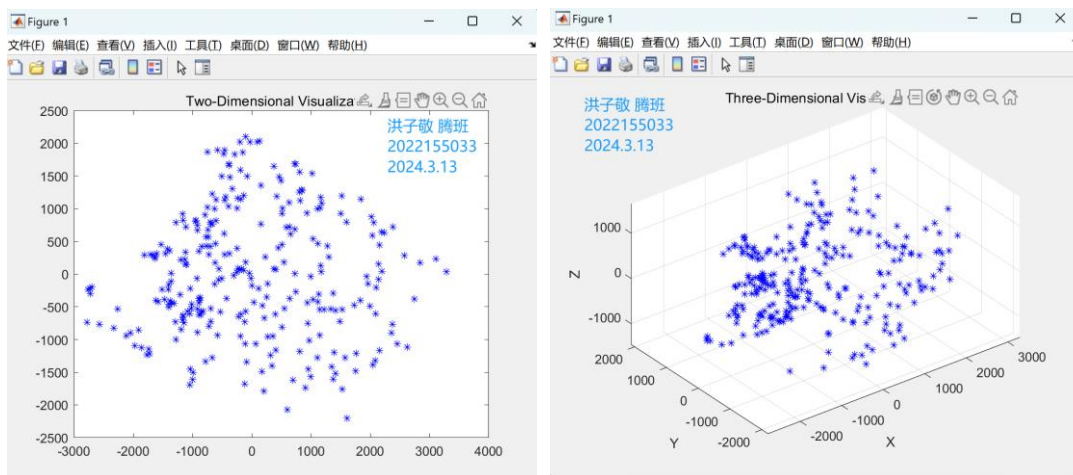
```
% visualize
fig=TwoDimensionVisualize(sorted_eig_vectors,X_train,mean);
waitfor(fig)
ThreeDimensionVisualize(sorted_eig_vectors,X_train,mean);
```

洪子敬 腾班
 2022155033
 2024.3.13

在旋转目标（COIL）数据集上二维，三维效果图展示如下所示：



在人脸（AR_Gray_50by40）数据集上二维，三维效果展示如下所示：



观察数据集在二维，三维的展示效果，我们可以看到样本点分布都比较分散，说明较好的保留了原有样本的方差。不难发现为什么在旋转目标（COIL）数据集上识别率较好，因为观察其二三维图像可以看到数据点有明显的分堆，这样识别起来会较其他样本更为简单，而对于人脸（AR_Gray_50by40）数据集，虽然它的数据点也没有那么集中，但相比前者还是集中了一些，这就是为什么后者识别率没有前者好的原因。

6. 传统 PCA 算法的改进

通过阅读相关博客和论文，学习到了一种改进的 PCA 算法—2DPCA。与传统 PCA 一样，都是最大化投影的协方差矩阵的迹（散度），而与传统 PCA 不同的是，2DPCA 直接使用原始图像数据矩阵进行处理，而非将其压缩到一维再进行降维。

用数学语言表示就是已知图像数据矩阵 A ，我们寻求一个标准正交列向量来实现以下的特征提取：

$$Y = AX$$

其中 Y 是二维图像投影之后得到的新特征。

为了使变化后投影特征的方差最大或是协方差矩阵的迹最大,我们需要最大化函数 $J(X) = \text{trace}(S)$,

$$\text{即 } \operatorname{argmax}_X \text{trace}(S) = X^T E[(A - \bar{A})^T (A - \bar{A})] X;$$

不妨定义图像协方差矩阵为

$$G = E[(A - \bar{A})^T (A - \bar{A})],$$

$$\text{则 } J(X) = X^T G X;$$

至此又回到了传统的 PCA 算法流程上,这里不多加赘述,其中要注意的是 $\bar{A}$ 表示的是平均图像, $\bar{A} = \frac{1}{N} \sum_{i=1}^N A_i$ 。

2DPCA 在图像数据中更好地保留了方差信息。传统 PCA 只考虑了数据在每个维度上的方差,而 2DPCA 同时考虑了行和列方向上的方差,更全面地捕捉了图像中的变化;除此之外 2DPCA 可以提取出与图像相关的特征,例如纹理、边缘和形状等。这使得它在图像处理和模式识别任务中更具优势。而传统 PCA 更适用于提取数值型数据的主要特征;值得注意的是 2DPCA 通常比传统 PCA 更高效。由于 2DPCA 考虑了数据的特殊结构,它可以通过利用矩阵操作的性质来减少计算量。

四、实验结论或体会

实验结论:

本次实验成功设计了 PCA 算法,并用其结合了最邻近算法进行了人脸识别和二三维度的可视化,取得了较好的结果,本次实验圆满结束。

实验体会:

本实验最大的收获就是基本掌握了 PCA 算法,从原理到每一步的实践,都自己一步步落实了,相比大一时候的懵懵懂懂还是进步了不少;除此之外也初步接触了矩阵的相关计算,明白了数据拉成列向量和行向量之间的区别,是左乘还是右乘,要不要转置,都比较清楚了。

此外,本实验采用了分函数多个进行编写,唯留主函数来编写代码框架,整个逻辑都十分清晰,每一步的思想都清晰可见,这个封装的思想也是第一次实践,感觉很有效果,实践出来也很有成就感。

另外实验过程中也发现自己很多不足的地方,比如数学方面线性代数关于矩阵的知识还得去深造一下,另外对改进 PCA 算法还没有储备很多的知识,暂时还没有想法涌现,仍需去多读一些论文来包装一下自己;当然算法在一些数据集上的识别率还不是很,还可以继续改进。前路漫漫,还得继续加油!

参考文献:

[1] [2DPCA—二维主成分分析详解及编程](#)

[2] PCA-Generalized 2D principal component analysis for face image representation 05NN

[3] [主成分分析 \(PCA\) 基于最大投影方差和最小重构距离_pca 的最近重构性](#)

指导教师批阅意见:

成绩评定:

指导教师签字:

年 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。